Project – High Level Design

On

Agriculture System Triage Agent

Course name : Agentic AI

Institution name: Medicaps University – Datagami skill based course

Student Name(s) & Enrollment Number(s)

Sr.no	Name	Enrollment Number
1	PRANITI KSHIRSAGAR	EN22CS301726
2	PRANJAL PATIDAR	EN22CS301730
3	PRAFULLA KUMAR GUPTA	EN22CS301715
4	PAWANI SHAH	EN22CS301700
5	RITIK THAKUR	EN22CS301811

Group Name: Group 08D7

Project Number: AAI-31

Industry Mentor Name: Aashruti Shah

University Mentor Name: Ajeet Singh Rajput

Academic Year: Jan 2026 – May 2026

1. Introduction

An **Agriculture Support Triage Agent** is an AI-based assistant that automatically reads farmers' messages and provides quick support. It understands the problem (such as crop disease, irrigation, fertilizer, or weather issues), determines how urgent the situation is, extracts important details like crop name and symptoms, and generates a helpful response. Urgent cases are forwarded to agricultural experts, while common queries receive instant automated guidance.
The system helps reduce response delays, supports agricultural officers, and improves crop productivity by giving farmers timely advice.

- **Scope of the Document**

This document describes the design and working of the Agriculture Support Triage Agent system. It covers:

• Functional behavior of the AI agent
• System workflow and processing stages
• Message classification and urgency detection
• Information extraction from farmer messages
• Automatic response generation
• Integration with backend services
• Limitations and assumptions

The document focuses on the software architecture and AI logic, not hardware or IoT sensor implementation.

- **Intended Audience**

This document is useful for the following readers:

Developers

- Understand system modules
- Implement APIs and AI pipeline

AI/ML Engineers

- Design classification and NER models
- Improve response generation quality

Agricultural Authorities

- Understand how automation helps farmer communication
- Plan deployment in helpline systems

Researchers/Students

- Learn practical implementation of AI agents in agriculture

- **System Overview**

The Agriculture Support Triage Agent processes farmer messages step-by-step through an intelligent pipeline.

Step 1 — Message Input

Farmers send messages through:

- Mobile app
- SMS/WhatsApp chatbot
- Web portal

Example message:

"My tomato plants have yellow leaves and spots after rain. What should I do?"

Step 2 — Intent Classification

The system identifies the purpose of the message:

Possible Intent	Meaning
Disease Issue	Crop infected
Irrigation Problem	Water shortage/excess
Fertilizer Advice	Nutrient guidance
Weather Concern	Rain/frost warning
Scheme Inquiry	Government benefits

Step 3 — Urgency Detection

The agent decides priority level:

Urgency	Actions
High	Immediate expert escalation
Medium	AI advice + queue review
Low	Automatic response

Example:

Pest outbreak → High urgency
Fertilizer suggestion → Medium urgency
General information → Low urgency

Step 4 — Information Extraction (NER)

- Extracts important details from farmer message
- Identifies crop name
- Detects symptoms described
- Recognizes location of farm

- Finds time or environmental condition (after rain, night, morning, etc.)
- Estimates severity of problem (mild / spreading / severe)

Step 5 — Response Generation

- Uses extracted information and detected intent
- Predicts likely cause of the issue
- Suggests immediate practical solution
- Provides preventive measures
- Advises contacting agriculture officer if problem continues

Step 6 — Routing Decision

- Severe or high-risk disease → forward to agriculture expert immediately
- Moderate issue → send AI advice and keep under monitoring
- General or informational query → automatic reply to farmer

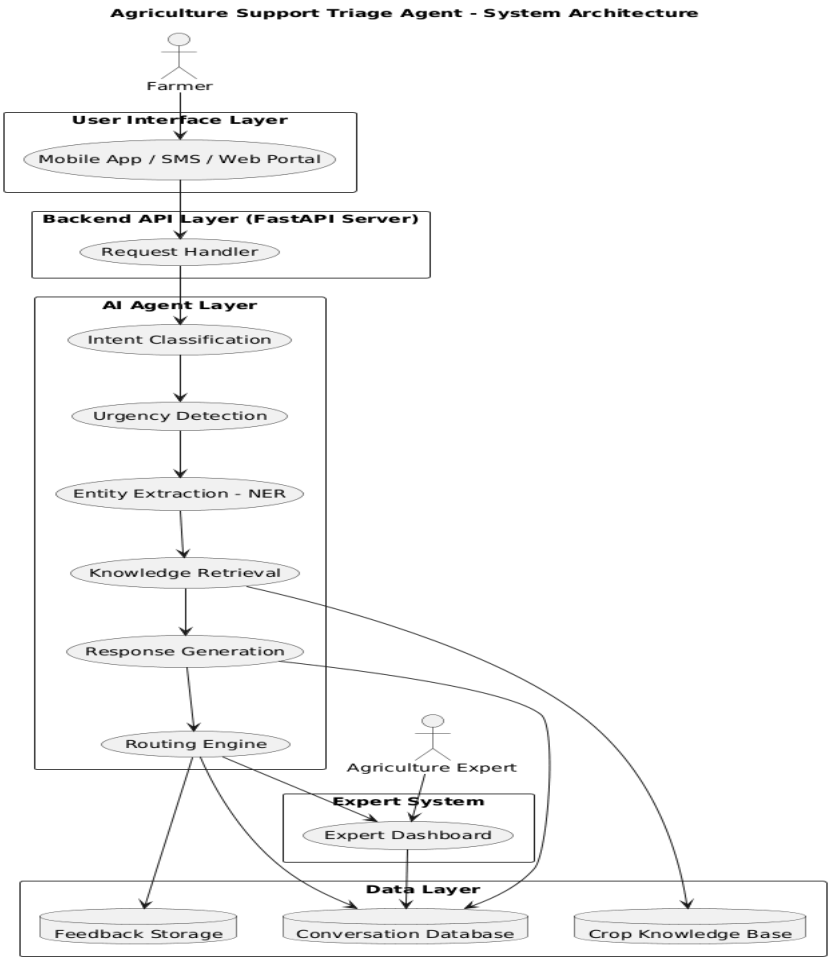
Step 7 — Continuous Learning

- Learns from expert corrections
- Learns from farmer feedback
- Uses historical solved cases
- Improves classification accuracy over time
- Improves response quality with more interactions

2. Design Application

2.1 Application Design

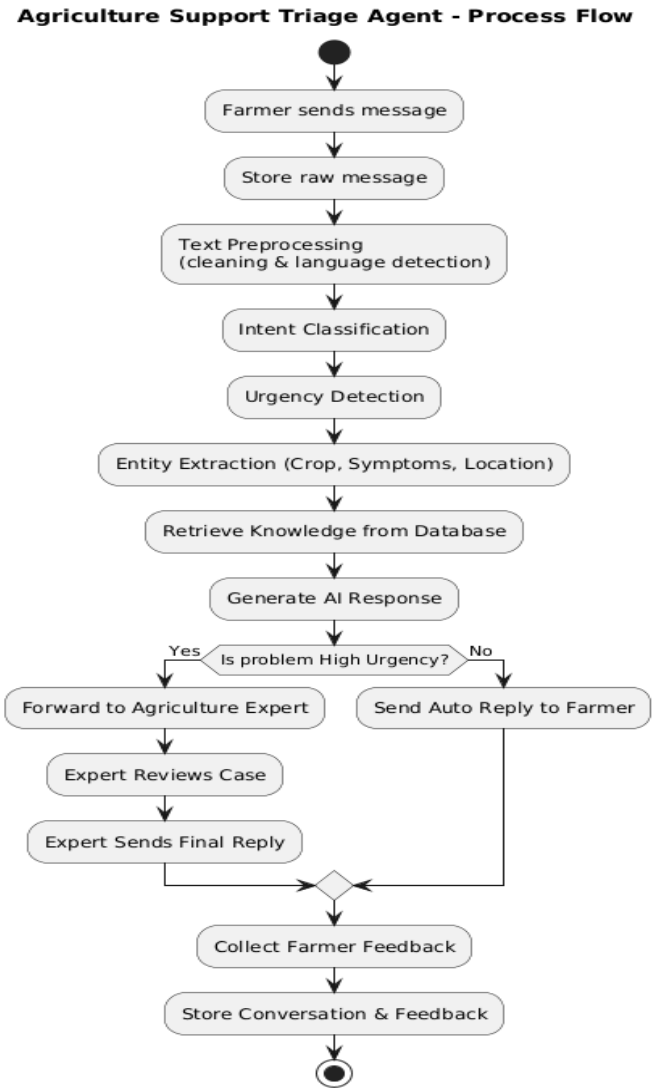
- Farmers send queries through app, SMS, or web interface
- Backend API receives and manages requests
- AI agent analyzes message and prepares reply
- Knowledge base provides agriculture information
- Critical cases are forwarded to agriculture experts
- Responses and feedback stored for improvement



2.2 Process Workflow

The message moves step-by-step through the system.

- Farmer submits message
- System stores raw message
- Language detection and cleaning performed
- AI identifies intent
- AI determines urgency
- System extracts crop, symptoms, location and time
- AI generates suggested solution
- Routing decision taken
- If urgent → send to human expert
- If normal → auto reply sent
- Feedback collected from farmer
- Data stored for learning



2.3 Information Flow

How data travels inside the system.

Input Flow

- Farmer message → API → Pre-processing

Processing Flow

- Cleaned text → Intent classifier
- Classified intent → Urgency predictor
- Text → Entity extractor
- Extracted data → Response generator

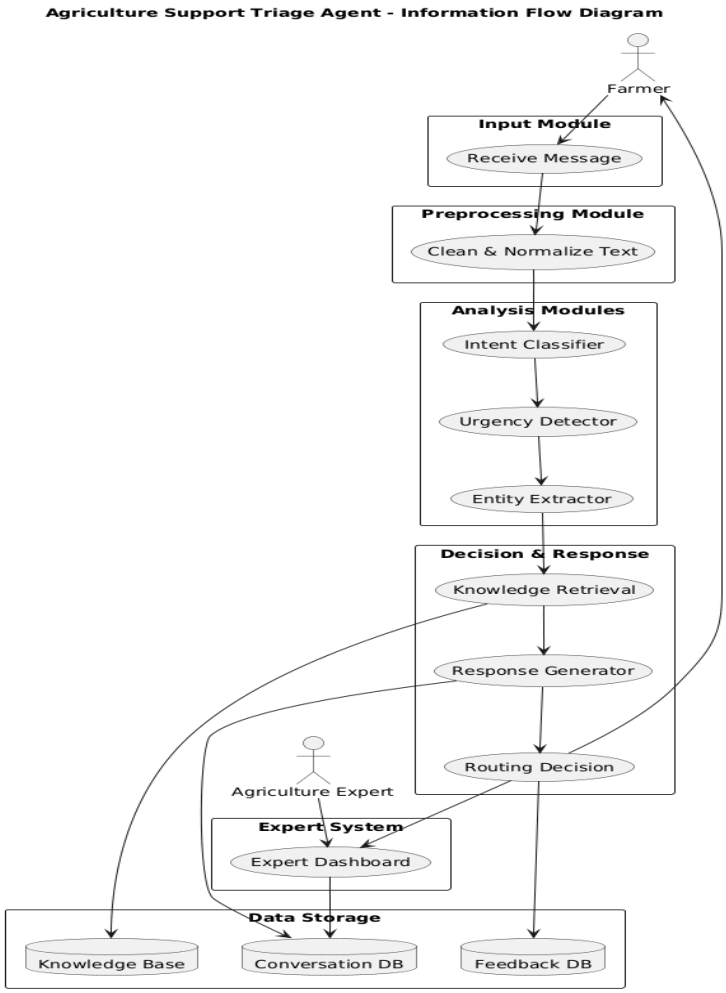
Decision Flow

- Urgency + intent → Routing engine

- Routing engine → AI reply OR Expert dashboard

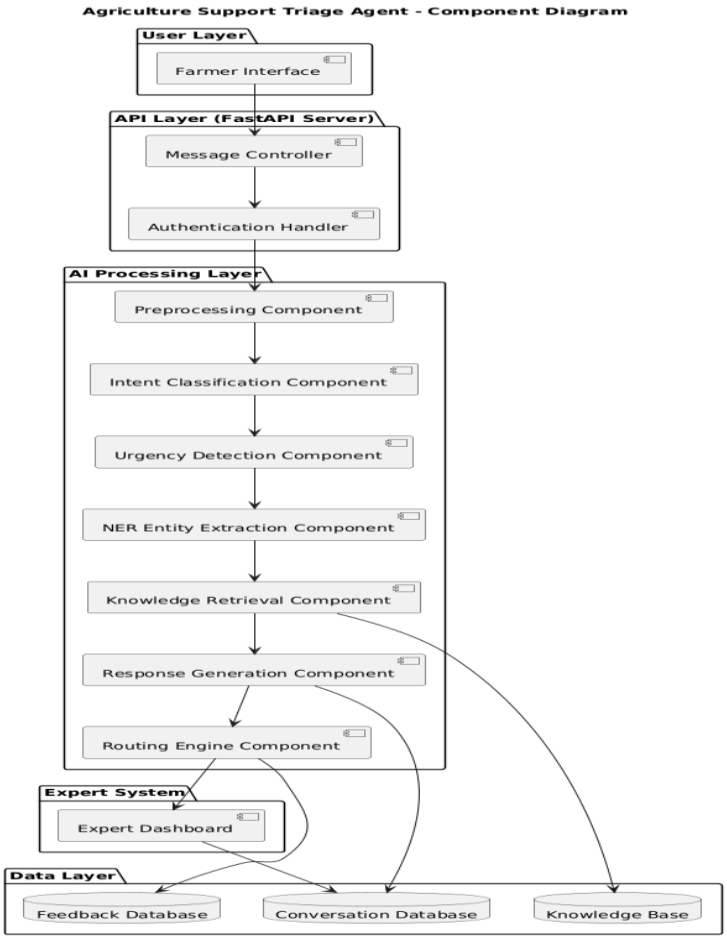
Output Flow

- Final response → Farmer
- Case stored → Database
- Feedback → Learning module



2.4 Components Design

- Message receiver collects user input
- Preprocessing prepares text
- Intent classifier identifies problem type
- Urgency detector finds priority level
- Entity extractor finds crop and symptoms
- Knowledge retrieval finds solution
- Response generator creates advice
- Routing engine decides action
- Expert dashboard handles complex cases
- Learning module improves system



2.5 Key Design Considerations

- Ensure accurate and safe advice
- Support low internet connectivity
- Handle multiple languages
- Scalable for many farmer
- Human involvement for risky cases
- Maintain data privacy

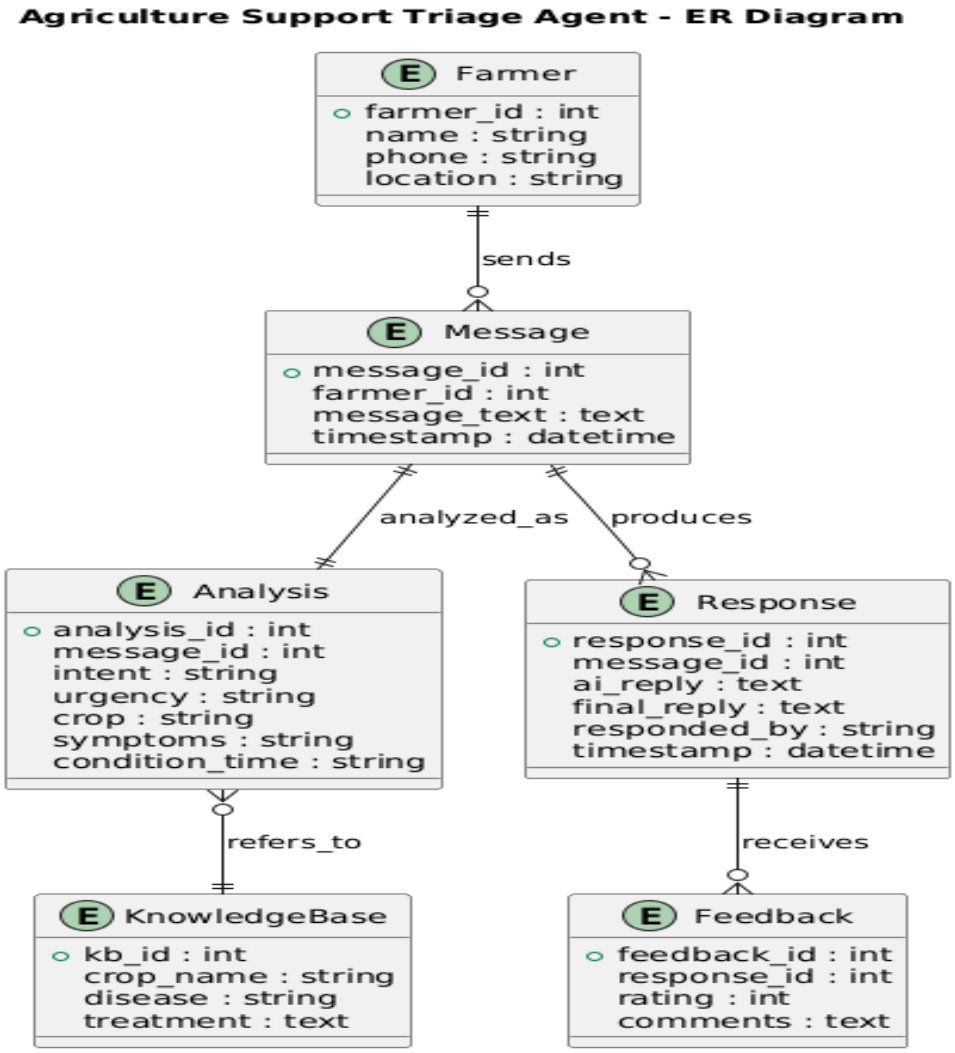
2.6 API Catalogue

- Send message API receives farmer query
- Analyze message API processes text
- Generate response API creates advice
- Routing decision API selects action
- Expert review API allows human response
- Feedback API collects ratings
- History API retrieves past conversations

3. Data Design

3.1 Data Model

- Stores farmer profile details such as name, contact and location
- Stores incoming messages and timestamps
- Stores detected intent and urgency level
- Stores extracted entities like crop, symptoms and conditions
- Stores AI generated responses
- Stores expert corrected responses
- Stores feedback ratings and comments
- Maintains history of previous conversations



3.2 Data Access Mechanism

- Backend API reads and writes data through database queries
- Only authenticated services can access stored data
- Experts access data through dashboard interface

- AI modules retrieve knowledge from knowledge base
- Logging system records each interaction for monitoring
- Role based access used for admin and experts

3.3 Data Retention Policies

- Conversation history stored for future learning
- Personal data stored only as long as required
- Old inactive records archived periodically
- Sensitive data protected and encrypted
- Farmers can request deletion of their data
- Feedback data retained for model improvement

3.4 Data Migration

- Existing farmer records can be imported from CSV or database
- Data format standardized before storage
- Backup created before migration
- Migration scripts verify data integrity
- System supports future database upgrades
- Old database data mapped to new schema without loss

4. Interfaces

4.1 User Interface (Farmer Side)

- Farmers enter queries through mobile app, SMS, WhatsApp or web portal
- Simple text input in local language supported
- Displays AI advice or expert reply
- Allows farmers to rate response usefulness
- Shows previous conversation history

4.2 Expert Interface (Agriculture Officer Dashboard)

- Experts view high-priority farmer cases
- Displays extracted crop, symptoms and urgency
- Experts edit or send final response
- Can mark case resolved or ongoing
- Provides analytics of reported issues

4.3 System Interface (Internal AI Modules)

- Preprocessing module sends cleaned text to classifier
- Classifier shares result with urgency detector

- Entity extractor passes data to response generator
- Routing module sends decision to notification service

4.4 External Interfaces

- Weather API provides climate data
- Government scheme database provides policy information
- SMS/WhatsApp gateway sends responses to farmers
- Database interface stores conversations and feedback

5. State and Session Management

5.1 Session Handling

- Each farmer interaction assigned a unique session ID
- Session starts when farmer sends a new message
- Session remains active for continuous conversation
- Inactive sessions automatically expire after timeout
- Same farmer messages grouped into one conversation

5.2 Conversation State

- System remembers previous questions in session
- Context used to generate better follow-up replies
- Tracks whether case is open, ongoing, or resolved
- Maintains escalation status (AI handled or expert handled)

5.3 User State Tracking

- Stores farmer interaction history
- Identifies repeat issues from same farmer
- Tracks feedback and satisfaction level
- Helps personalize future responses

5.4 Session Storage

- Session data stored in database or cache
- Quick retrieval for ongoing conversations
- Cleared after long inactivity period
- Backup stored for learning and analytics

5.5 Error and Recovery Handling

- System resumes conversation after temporary failure
- Prevents duplicate responses
- Logs incomplete sessions for review
- Ensures message delivery confirmation

6. Caching

6.1 Purpose of Caching

- Provide instant replies for repeated farmer questions
- Reduce AI model processing time
- Save API usage cost
- Support farmers in low-network areas
- Improve overall system performance

6.2 Cached Data Types

- Common crop disease questions and solutions
- Frequently generated AI responses
- Recent weather queries by location
- Government scheme information
- Farmer session conversation context

6.3 Cache Storage

- Stored in fast memory cache (e.g., Redis)
- Separate from main database storage
- Accessed before running AI processing
- Automatically synchronized with database

6.4 Cache Expiry Policy

- Weather data expires quickly
- Disease treatment advice refreshed periodically
- Static knowledge stored longer
- Cache cleared when updated by experts

6.5 Cache Update Strategy

- Update cache after expert correction
- Replace outdated agricultural recommendations
- Refresh frequently used queries first
- Remove unused data after timeout

6.6 Cache Flow in System

- System first checks cache for similar query
- If found → instant response returned
- If not found → AI processes message
- Result stored in cache for future reuse

7. Non-functional requirements

Non-functional requirements describe how well the Agriculture Support Triage Agent system works, rather than what it does. They define qualities like security, speed, reliability, scalability, and usability to ensure the system is safe, fast, and dependable for farmers and agricultural experts.

7.1 Security Aspects

- User authentication for experts and admin access
- Role-based authorization to restrict sensitive operations
- Encryption of farmer personal data during storage and transfer
- Secure API communication using HTTPS
- Protection against spam and malicious inputs
- Input validation to prevent injection attacks
- Access logs maintained for auditing
- Data privacy maintained according to policies
- Regular backup to prevent data loss
- Secure storage of API keys and credentials

7.2 Performance Aspects

- System responds within a few seconds for normal queries
- Handles multiple farmer requests simultaneously
- Uses caching to speed up repeated responses
- Scalable backend to support large number of users
- Lightweight responses for low internet connectivity
- Background processing for heavy AI tasks
- Efficient database indexing for fast retrieval
- Graceful handling during peak usage
- Automatic retry in case of temporary failure

8. Refrences

- FastAPI Official Documentation — API development and backend request handling
- Redis Documentation — Caching and session storage concepts
- PlantUML Documentation — Architecture and design diagram creation
- Natural Language Processing concepts — Intent classification and Named Entity Recognition (NER)
- Large Language Model (LLM) based response generation — Automated reply generation
- Agent-based system design principles — Triage decision and routing mechanism
- Database design concepts — ER modeling, data storage, and retrieval
- Web API integration concepts — External services like weather and messaging gateways